VIT – School of Computer Science Engineering and Information Systems

BCSE302L – Database Systems | Course Project

REVIEW I – Problem Analysis & Database Design

Project Title: Smart Hospital Management System with Emergency Triage, Bed Allocation and Audit Logging

Team Members

S.No.	Name	Register Number
1	Anushka Jaiswal	24BYB0121
2	Shubham Mishra	24BYB0117

1. DDL and Constraints

1.1 Table Creation

```
SQL>
SQL> PROMPT >>> PATIENT
>>> PATIENT
SQL> CREATE TABLE patient (
  2    patient_id         NUMBER        NOT NULL,
  3    patient_name       VARCHAR2(50)  NOT NULL,
  4    gender             VARCHAR2(10),
  5    date_of_birth      DATE          NOT NULL,
  6    phone              VARCHAR2(15)  NOT NULL,
  7    address            VARCHAR2(100),
  8    blood_group        VARCHAR2(5),
  9    emergency_contact  VARCHAR2(15),
 10    registration_date  DATE          DEFAULT SYSDATE,
 11    CONSTRAINT pk_patient     PRIMARY KEY (patient_id),
 12    CONSTRAINT uq_pat_phone   UNIQUE (phone),
 13    CONSTRAINT chk_pat_gender CHECK (gender IN ('Male','Female','Other')),
 14    CONSTRAINT chk_pat_phone  CHECK (REGEXP_LIKE(phone, '^[0-9]{10}$')),
 15    CONSTRAINT chk_pat_blood  CHECK (blood_group IN
 16                              ('A+','A-','B+','B-','AB+','AB-','O+','O-'))
 17  );

Table created.
```

Figure 1: *CREATE TABLE patient – primary key, unique phone, gender/phone/blood-group checks*

```
SQL>
SQL> PROMPT >>> DOCTOR
>>> DOCTOR
SQL> CREATE TABLE doctor (
  2    doctor_id            NUMBER        NOT NULL,
  3    doctor_name          VARCHAR2(50)  NOT NULL,
  4    department           VARCHAR2(30)  NOT NULL,
  5    specialization       VARCHAR2(40),
  6    phone                VARCHAR2(15),
  7    consultation_fee     NUMBER(8,2)   NOT NULL,
  8    shift                VARCHAR2(10),
  9    availability_status  VARCHAR2(10)  DEFAULT 'Available' NOT NULL,
 10    CONSTRAINT pk_doctor      PRIMARY KEY (doctor_id),
 11    CONSTRAINT uq_doc_phone   UNIQUE (phone),
 12    CONSTRAINT chk_doc_fee    CHECK (consultation_fee > 0),
 13    CONSTRAINT chk_doc_shift  CHECK (shift IN ('Morning','Evening','Night')),
 14    CONSTRAINT chk_doc_avail  CHECK (availability_status IN ('Available','Busy'))
 15  );

Table created.
```

Figure 2: *CREATE TABLE doctor – unique phone, fee > 0, shift and availability checks*

```
SQL>
SQL> PROMPT >>> BED
>>> BED
SQL> CREATE TABLE bed (
  2    bed_id        NUMBER       NOT NULL,
  3    ward_type     VARCHAR2(15) NOT NULL,
  4    bed_status    VARCHAR2(10) DEFAULT 'Free' NOT NULL,
  5    daily_charge  NUMBER(8,2)  NOT NULL,
  6    CONSTRAINT pk_bed        PRIMARY KEY (bed_id),
  7    CONSTRAINT chk_bed_ward  CHECK (ward_type IN
  8                             ('General','Private','ICU','Emergency')),
  9    CONSTRAINT chk_bed_stat  CHECK (bed_status IN ('Free','Occupied','Cleaning')),
 10    CONSTRAINT chk_bed_chg   CHECK (daily_charge > 0)
 11  );

Table created.

SQL>
SQL> PROMPT >>> MEDICINE
>>> MEDICINE
SQL> CREATE TABLE medicine (
  2    medicine_id     NUMBER        NOT NULL,
  3    medicine_name   VARCHAR2(50)  NOT NULL,
  4    manufacturer    VARCHAR2(50),
  5    price_per_unit  NUMBER(8,2)   NOT NULL,
  6    stock_quantity  NUMBER        DEFAULT 0 NOT NULL,
  7    CONSTRAINT pk_medicine    PRIMARY KEY (medicine_id),
  8    CONSTRAINT chk_med_price  CHECK (price_per_unit > 0),
  9    CONSTRAINT chk_med_stock  CHECK (stock_quantity >= 0)
 10  );

Table created.
```

Figure 3: *CREATE TABLE bed and medicine – ward/status checks, positive charge and price, non-negative stock*

```
SQL>
SQL> PROMPT >>> VISIT
>>> VISIT
SQL> CREATE TABLE visit (
  2    visit_id        NUMBER        NOT NULL,
  3    patient_id      NUMBER        NOT NULL,
  4    doctor_id       NUMBER        NOT NULL,
  5    bed_id          NUMBER,
  6    visit_type      VARCHAR2(12)  NOT NULL,
  7    visit_datetime  TIMESTAMP     NOT NULL,
  8    complaint       VARCHAR2(100),
  9    triage_level    NUMBER(1),
 10    visit_status    VARCHAR2(15)  DEFAULT 'Scheduled' NOT NULL,
 11    admit_time      TIMESTAMP,
 12    discharge_time  TIMESTAMP,
 13    CONSTRAINT pk_visit          PRIMARY KEY (visit_id),
 14    CONSTRAINT fk_visit_patient  FOREIGN KEY (patient_id)
 15                                 REFERENCES patient (patient_id),
 16    CONSTRAINT fk_visit_doctor   FOREIGN KEY (doctor_id)
 17                                 REFERENCES doctor (doctor_id),
 18    CONSTRAINT fk_visit_bed      FOREIGN KEY (bed_id)
 19                                 REFERENCES bed (bed_id),
 20    CONSTRAINT chk_visit_type    CHECK (visit_type IN ('Appointment','Emergency')),
 21    CONSTRAINT chk_visit_status  CHECK (visit_status IN ('Scheduled','Waiting',
 22                                 'Under Treatment','Admitted','Discharged',
 23                                 'Cancelled')),
 24    CONSTRAINT chk_visit_range   CHECK (triage_level BETWEEN 1 AND 4),
 25    CONSTRAINT chk_visit_triage  CHECK (
 26                   (visit_type = 'Emergency'   AND triage_level IS NOT NULL) OR
 27                   (visit_type = 'Appointment' AND triage_level IS NULL)),
 28    CONSTRAINT chk_visit_bed     CHECK (visit_status <> 'Admitted'
 29                                 OR bed_id IS NOT NULL),
 30    CONSTRAINT chk_visit_disch   CHECK (discharge_time >= admit_time)
 31  );

Table created.
```

Figure 4: *CREATE TABLE visit – three foreign keys with triage, admission-bed and discharge-time rules*

```
SQL>
SQL> PROMPT >>> PRESCRIPTION
>>> PRESCRIPTION
SQL> CREATE TABLE prescription (
  2    prescription_id  NUMBER        NOT NULL,
  3    visit_id         NUMBER        NOT NULL,
  4    medicine_id      NUMBER        NOT NULL,
  5    dosage           VARCHAR2(30),
  6    duration_days    NUMBER,
  7    quantity         NUMBER,
  8    CONSTRAINT pk_prescription  PRIMARY KEY (prescription_id),
  9    CONSTRAINT fk_presc_visit   FOREIGN KEY (visit_id)
 10                               REFERENCES visit (visit_id) ON DELETE CASCADE,
 11    CONSTRAINT fk_presc_med     FOREIGN KEY (medicine_id)
 12                               REFERENCES medicine (medicine_id),
 13    CONSTRAINT chk_presc_days   CHECK (duration_days > 0),
 14    CONSTRAINT chk_presc_qty    CHECK (quantity > 0)
 15  );

Table created.
```

Figure 5: *CREATE TABLE prescription – foreign keys to visit (ON DELETE CASCADE) and medicine*

```
SQL>
SQL> PROMPT >>> BILL
>>> BILL
SQL> CREATE TABLE bill (
  2    bill_id              NUMBER        NOT NULL,
  3    visit_id             NUMBER        NOT NULL,
  4    consultation_charge  NUMBER(8,2)   DEFAULT 0 NOT NULL,
  5    medicine_charge      NUMBER(8,2)   DEFAULT 0 NOT NULL,
  6    bed_charge           NUMBER(8,2)   DEFAULT 0 NOT NULL,
  7    total_amount         NUMBER(10,2)  NOT NULL,
  8    payment_mode         VARCHAR2(10),
  9    payment_status       VARCHAR2(10)  DEFAULT 'Pending' NOT NULL,
 10    bill_date            DATE          DEFAULT SYSDATE,
 11    CONSTRAINT pk_bill         PRIMARY KEY (bill_id),
 12    CONSTRAINT uq_bill_visit   UNIQUE (visit_id),
 13    CONSTRAINT fk_bill_visit   FOREIGN KEY (visit_id)
 14                               REFERENCES visit (visit_id),
 15    CONSTRAINT chk_bill_mode   CHECK (payment_mode IN ('Cash','Card','UPI')),
 16    CONSTRAINT chk_bill_stat   CHECK (payment_status IN ('Paid','Pending')),
 17    CONSTRAINT chk_bill_amt    CHECK (consultation_charge >= 0
 18                               AND medicine_charge >= 0 AND bed_charge >= 0),
 19    CONSTRAINT chk_bill_total  CHECK (total_amount = consultation_charge
 20                               + medicine_charge + bed_charge),
 21    CONSTRAINT chk_bill_paid   CHECK (payment_status = 'Pending'
 22                               OR payment_mode IS NOT NULL)
 23  );

Table created.
```

Figure 6: *CREATE TABLE bill – one bill per visit; total must equal the sum of charges; Paid requires a payment mode*

```
SQL>
SQL> PROMPT >>> AUDIT_LOG
>>> AUDIT_LOG
SQL> CREATE TABLE audit_log (
  2    log_id       NUMBER         NOT NULL,
  3    table_name   VARCHAR2(30)   NOT NULL,
  4    action_type  VARCHAR2(10)   NOT NULL,
  5    record_id    NUMBER,
  6    changed_by   VARCHAR2(30)   DEFAULT USER,
  7    change_time  TIMESTAMP      DEFAULT SYSTIMESTAMP,
  8    description  VARCHAR2(200),
  9    CONSTRAINT pk_audit_log   PRIMARY KEY (log_id),
 10    CONSTRAINT chk_audit_act  CHECK (action_type IN ('INSERT','UPDATE','DELETE'))
 11  );

Table created.
```

Figure 7: *CREATE TABLE audit_log – action type restricted to INSERT, UPDATE or DELETE*

1.2 Indexes and Sequences

```
SQL> CREATE UNIQUE INDEX uq_doctor_slot ON visit (
  2    CASE WHEN visit_status <> 'Cancelled' THEN doctor_id END,
  3    CASE WHEN visit_status <> 'Cancelled' THEN visit_datetime END
  4  );

Index created.
```

Figure 8: *Unique index uq_doctor_slot – a doctor cannot have two active visits at the same date and time*

```
SQL> PROMPT >>> Unique index: a bed cannot be held by two admitted patients at once
>>> Unique index: a bed cannot be held by two admitted patients at once
SQL> CREATE UNIQUE INDEX uq_active_bed ON visit (
  2    CASE WHEN visit_status = 'Admitted' THEN bed_id END
  3  );

Index created.
```

Figure 9: *Unique index uq_active_bed – a bed cannot be held by two admitted patients at once*

```
SQL> PROMPT >>> Indexes on the foreign keys used most in joins
>>> Indexes on the foreign keys used most in joins
SQL> CREATE INDEX idx_visit_patient ON visit (patient_id);

Index created.
```

Figure 10: *Index idx_visit_patient on the most-used join column*

```
SQL> CREATE INDEX idx_visit_doctor  ON visit (doctor_id);

Index created.
```

Figure 11: *Index idx_visit_doctor on the doctor foreign key*

```
SQL> PROMPT >>> Sequences for the primary keys (start values continue after the sample data)
>>> Sequences for the primary keys (start values continue after the sample data)
SQL> CREATE SEQUENCE seq_patient       START WITH 113 INCREMENT BY 1 NOCACHE;

Sequence created.
```

Figure 12: *Sequence seq_patient (starts at 113)*

```
SQL> CREATE SEQUENCE seq_visit         START WITH 421 INCREMENT BY 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_prescription  START WITH 623 INCREMENT BY 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_bill          START WITH 712 INCREMENT BY 1 NOCACHE;

Sequence created.

SQL> CREATE SEQUENCE seq_audit         START WITH 13  INCREMENT BY 1 NOCACHE;

Sequence created.
```

Figure 13: *Sequences seq_visit, seq_prescription, seq_bill and seq_audit*

2. DML and Data Population

2.1 Sample Data Insertion

```
SQL> PROMPT >>> PATIENT (12 rows)
>>> PATIENT (12 rows)
SQL> INSERT INTO patient VALUES (101, 'Arjun Reddy', 'Male', DATE '1985-03-14',
  2    '9876500101', '12 Gandhi Road, Chennai', 'B+', '9876511101', DATE '2026-01-10');

1 row created.

SQL> INSERT INTO patient VALUES (102, 'Kavita Sharma', 'Female', DATE '1972-07-22',
  2    '9876500102', '45 MG Road, Bengaluru', 'O+', '9876511102', DATE '2026-01-18');

1 row created.

SQL> INSERT INTO patient VALUES (103, 'Rahul Verma', 'Male', DATE '1990-11-05',
  2    '9876500103', '7 Lake View, Hyderabad', 'A+', '9876511103', DATE '2026-02-02');

1 row created.

SQL> INSERT INTO patient VALUES (104, 'Divya Krishnan', 'Female', DATE '1998-01-30',
  2    '9876500104', '23 Anna Nagar, Chennai', 'AB+', '9876511104', DATE '2026-02-14');

1 row created.

SQL> INSERT INTO patient VALUES (105, 'Mohammed Irfan', 'Male', DATE '2018-05-19',
  2    '9876500105', '18 Park Street, Kolkata', 'O-', '9876511105', DATE '2026-03-05');

1 row created.

SQL> INSERT INTO patient VALUES (106, 'Sunita Patil', 'Female', DATE '1965-09-09',
  2    '9876500106', '5 FC Road, Pune', 'B-', '9876511106', DATE '2026-03-21');

1 row created.

SQL> INSERT INTO patient VALUES (107, 'Aakash Gupta', 'Male', DATE '2001-12-12',
  2    '9876500107', '88 Civil Lines, Delhi', 'A-', '9876511107', DATE '2026-04-11');

1 row created.

SQL> INSERT INTO patient VALUES (108, 'Lakshmi Narayanan', 'Female', DATE '1958-02-27',
  2    '9876500108', '9 Temple Street, Madurai', 'O+', '9876511108', DATE '2026-05-02');

1 row created.
```

Figure 14: *INSERT statements for PATIENT (records 101–105)*

```
SQL> INSERT INTO patient VALUES (109, 'Vivek Joshi', 'Male', DATE '1979-06-18',
  2    '9876500109', '31 Residency Road, Bengaluru', 'B+', '9876511109',
  3    DATE '2026-06-15');

1 row created.

SQL> INSERT INTO patient VALUES (110, 'Neha Kapoor', 'Female', DATE '1988-04-03',
  2    '9876500110', '14 Sector 21, Noida', 'A+', '9876511110', DATE '2026-07-08');

1 row created.

SQL> INSERT INTO patient VALUES (111, 'Imran Khan', 'Male', DATE '1995-08-25',
  2    '9876500111', '2 Mount Road, Chennai', 'AB-', '9876511111', DATE '2026-09-14');

1 row created.

SQL> INSERT INTO patient VALUES (112, 'Pooja Menon', 'Female', DATE '1993-12-01',
  2    '9876500112', '6 Marine Drive, Kochi', 'O+', '9876511112', DATE '2026-09-15');

1 row created.

SQL>
SQL> PROMPT >>> DOCTOR (10 rows)
>>> DOCTOR (10 rows)
SQL> INSERT INTO doctor VALUES (201, 'Dr. Rajesh Kumar', 'Cardiology', 'Cardiologist',
  2    '9840011101', 800, 'Morning', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (202, 'Dr. Priya Nair', 'General Medicine', 'Physician',
  2    '9840011102', 500, 'Morning', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (203, 'Dr. Arvind Menon', 'Emergency',
  2    'Emergency Physician', '9840011103', 1000, 'Night', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (204, 'Dr. Sneha Iyer', 'Orthopedics',
  2    'Orthopedic Surgeon', '9840011104', 700, 'Evening', 'Available');

1 row created.
```

Figure 15: *INSERT statements for PATIENT (records 109–112) and the start of DOCTOR*

```
SQL> INSERT INTO doctor VALUES (205, 'Dr. Karthik Raman', 'Neurology', 'Neurologist',
  2    '9840011105', 900, 'Morning', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (206, 'Dr. Fatima Sheikh', 'Pediatrics', 'Pediatrician',
  2    '9840011106', 600, 'Morning', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (207, 'Dr. Vikram Singh', 'Emergency', 'Trauma Specialist',
  2    '9840011107', 1000, 'Evening', 'Busy');

1 row created.

SQL> INSERT INTO doctor VALUES (208, 'Dr. Meena Pillai', 'Gynecology', 'Gynecologist',
  2    '9840011108', 750, 'Evening', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (209, 'Dr. Suresh Babu', 'General Medicine', 'Physician',
  2    '9840011109', 500, 'Evening', 'Available');

1 row created.

SQL> INSERT INTO doctor VALUES (210, 'Dr. Anita Desai', 'Dermatology', 'Dermatologist',
  2    '9840011110', 550, 'Morning', 'Available');

1 row created.
```

Figure 16: *INSERT statements for DOCTOR (records 205–210)*

```
SQL> PROMPT >>> BED (12 rows)
>>> BED (12 rows)
SQL> INSERT INTO bed VALUES (301, 'General', 'Free', 600);

1 row created.

SQL> INSERT INTO bed VALUES (302, 'General', 'Free', 600);

1 row created.

SQL> INSERT INTO bed VALUES (303, 'General', 'Free', 600);

1 row created.

SQL> INSERT INTO bed VALUES (304, 'General', 'Cleaning', 600);

1 row created.

SQL> INSERT INTO bed VALUES (305, 'Private', 'Occupied', 2500);

1 row created.

SQL> INSERT INTO bed VALUES (306, 'Private', 'Free', 2500);

1 row created.

SQL> INSERT INTO bed VALUES (307, 'ICU', 'Free', 5000);

1 row created.

SQL> INSERT INTO bed VALUES (308, 'ICU', 'Occupied', 5000);

1 row created.

SQL> INSERT INTO bed VALUES (309, 'Emergency', 'Free', 1500);

1 row created.

SQL> INSERT INTO bed VALUES (310, 'Emergency', 'Free', 1500);

1 row created.

SQL> INSERT INTO bed VALUES (311, 'Emergency', 'Free', 1500);

1 row created.
```

Figure 17: *INSERT statements for BED (beds 301–311)*

```
SQL> INSERT INTO bed VALUES (312, 'Emergency', 'Cleaning', 1500);

1 row created.
```

Figure 18: *INSERT statement for BED (bed 312)*

```
SQL> PROMPT >>> MEDICINE (12 rows)
>>> MEDICINE (12 rows)
SQL> INSERT INTO medicine VALUES (501, 'Paracetamol 500mg', 'Cipla', 2.00, 480);

1 row created.

SQL> INSERT INTO medicine VALUES (502, 'Amoxicillin 500mg', 'Sun Pharma', 8.50, 276);

1 row created.

SQL> INSERT INTO medicine VALUES (503, 'Cetirizine 10mg', 'Dr. Reddys', 3.00, 388);

1 row created.

SQL> INSERT INTO medicine VALUES (504, 'Ibuprofen 400mg', 'Abbott', 4.50, 311);

1 row created.

SQL> INSERT INTO medicine VALUES (505, 'Pantoprazole 40mg', 'Cipla', 6.00, 238);

1 row created.

SQL> INSERT INTO medicine VALUES (506, 'Aspirin 75mg', 'Bayer', 1.50, 550);

1 row created.

SQL> INSERT INTO medicine VALUES (507, 'Atorvastatin 10mg', 'Lupin', 9.00, 160);

1 row created.

SQL> INSERT INTO medicine VALUES (508, 'Metformin 500mg', 'USV', 2.50, 270);

1 row created.

SQL> INSERT INTO medicine VALUES (509, 'Azithromycin 500mg', 'Alkem', 22.00, 8);

1 row created.

SQL> INSERT INTO medicine VALUES (510, 'Sumatriptan 50mg', 'Sun Pharma', 35.00, 5);

1 row created.

SQL> INSERT INTO medicine VALUES (511, 'Calcium + Vitamin D3', 'Torrent', 7.00, 120);

1 row created.

SQL> INSERT INTO medicine VALUES (512, 'ORS Sachet', 'FDC', 12.00, 0);

1 row created.
```

Figure 19: *INSERT statements for MEDICINE*

```
SQL> PROMPT >>> VISIT (20 rows: 13 appointments + emergency cases)
>>> VISIT (20 rows: 13 appointments + emergency cases)
SQL> INSERT INTO visit VALUES (401, 107, 210, NULL, 'Appointment',
  2    TIMESTAMP '2026-09-20 10:30:00', 'Skin rash', NULL, 'Discharged', NULL, NULL);

1 row created.

SQL> INSERT INTO visit VALUES (402, 108, 209, NULL, 'Appointment',
  2    TIMESTAMP '2026-09-22 17:00:00', 'Diabetes review', NULL, 'Discharged', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (403, 110, 202, NULL, 'Appointment',
  2    TIMESTAMP '2026-09-25 09:45:00', 'Persistent cough', NULL, 'Discharged', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (404, 109, 204, NULL, 'Appointment',
  2    TIMESTAMP '2026-09-28 18:00:00', 'Lower back pain', NULL, 'Discharged', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (405, 101, 202, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-01 10:00:00', 'Fever and cold', NULL, 'Discharged', NULL, NULL);

1 row created.

SQL> INSERT INTO visit VALUES (406, 102, 201, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-01 11:30:00', 'Chest discomfort', NULL, 'Discharged', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (407, 103, 203, 307, 'Emergency',
  2    TIMESTAMP '2026-10-02 22:15:00', 'Road accident - head injury', 1, 'Discharged',
  3    TIMESTAMP '2026-10-02 22:40:00', TIMESTAMP '2026-10-05 11:00:00');

1 row created.

SQL> INSERT INTO visit VALUES (408, 104, 204, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-03 16:00:00', 'Knee pain', NULL, 'Discharged', NULL, NULL);

1 row created.
```

Figure 20: *INSERT statements for VISIT – appointments 401–405*

```
SQL> INSERT INTO visit VALUES (409, 105, 206, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-04 09:30:00', 'Child fever', NULL, 'Discharged', NULL, NULL);

1 row created.

SQL> INSERT INTO visit VALUES (410, 106, 207, 310, 'Emergency',
  2    TIMESTAMP '2026-10-06 18:20:00', 'Fracture after fall from stairs', 2,
  3    'Discharged', TIMESTAMP '2026-10-06 19:00:00', TIMESTAMP '2026-10-08 12:00:00');

1 row created.

SQL> INSERT INTO visit VALUES (411, 107, 205, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-07 11:00:00', 'Recurring migraine', NULL, 'Discharged', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (412, 108, 208, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-08 15:30:00', 'Routine check-up', NULL, 'Cancelled', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (413, 109, 201, 305, 'Appointment',
  2    TIMESTAMP '2026-10-09 10:00:00', 'Chest pain - observation', NULL, 'Admitted',
  3    TIMESTAMP '2026-10-09 12:00:00', NULL);

1 row created.

SQL> INSERT INTO visit VALUES (414, 110, 203, 308, 'Emergency',
  2    TIMESTAMP '2026-10-10 02:30:00', 'Severe breathlessness', 1, 'Admitted',
  3    TIMESTAMP '2026-10-10 03:00:00', NULL);

1 row created.

SQL> INSERT INTO visit VALUES (415, 111, 207, NULL, 'Emergency',
  2    TIMESTAMP '2026-10-12 20:10:00', 'Deep cut on hand', 3, 'Under Treatment', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (416, 112, 203, NULL, 'Emergency',
  2    TIMESTAMP '2026-10-12 20:25:00', 'High fever and vomiting', 2, 'Waiting', NULL,
  3    NULL);

1 row created.
```

Figure 21: *INSERT statements for VISIT – visits 409–413 including an emergency admission and a cancellation*

```
SQL> INSERT INTO visit VALUES (417, 103, 207, NULL, 'Emergency',
  2    TIMESTAMP '2026-10-12 20:40:00', 'Minor ankle sprain', 4, 'Waiting', NULL, NULL);

1 row created.

SQL> INSERT INTO visit VALUES (418, 104, 203, NULL, 'Emergency',
  2    TIMESTAMP '2026-10-12 20:50:00', 'Severe allergic reaction', 1, 'Waiting', NULL,
  3    NULL);

1 row created.

SQL> INSERT INTO visit VALUES (419, 105, 202, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-14 10:00:00', 'Follow-up visit', NULL, 'Scheduled', NULL, NULL);

1 row created.

SQL> INSERT INTO visit VALUES (420, 102, 201, NULL, 'Appointment',
  2    TIMESTAMP '2026-10-15 11:00:00', 'ECG follow-up', NULL, 'Scheduled', NULL, NULL);

1 row created.
```

Figure 22: *INSERT statements for VISIT – waiting emergency cases and scheduled appointments (417–420)*

```
SQL> PROMPT >>> PRESCRIPTION (22 rows)
>>> PRESCRIPTION (22 rows)
SQL> INSERT INTO prescription VALUES (601, 401, 503, '0-0-1', 7, 7);

1 row created.

SQL> INSERT INTO prescription VALUES (602, 402, 508, '1-0-1', 30, 30);

1 row created.

SQL> INSERT INTO prescription VALUES (603, 403, 509, '1-0-0', 3, 3);

1 row created.

SQL> INSERT INTO prescription VALUES (604, 403, 501, '1-0-1', 3, 6);

1 row created.

SQL> INSERT INTO prescription VALUES (605, 404, 504, '1-0-1', 5, 10);

1 row created.

SQL> INSERT INTO prescription VALUES (606, 405, 501, '1-1-1', 3, 10);

1 row created.

SQL> INSERT INTO prescription VALUES (607, 405, 503, '0-0-1', 5, 5);

1 row created.

SQL> INSERT INTO prescription VALUES (608, 406, 506, '1-0-0', 30, 30);

1 row created.

SQL> INSERT INTO prescription VALUES (609, 406, 507, '0-0-1', 30, 30);

1 row created.

SQL> INSERT INTO prescription VALUES (610, 407, 502, '1-0-1', 7, 14);

1 row created.

SQL> INSERT INTO prescription VALUES (611, 407, 505, '1-0-0', 7, 7);

1 row created.

SQL> INSERT INTO prescription VALUES (612, 408, 504, '1-1-1', 5, 15);

1 row created.
```

Figure 23: *INSERT statements for PRESCRIPTION (601–609)*

```
SQL> INSERT INTO prescription VALUES (613, 408, 505, '1-0-0', 5, 5);

1 row created.

SQL> INSERT INTO prescription VALUES (614, 409, 501, '1-1-1', 3, 9);

1 row created.

SQL> INSERT INTO prescription VALUES (615, 410, 504, '1-0-1', 7, 14);

1 row created.

SQL> INSERT INTO prescription VALUES (616, 410, 511, '0-1-0', 30, 30);

1 row created.

SQL> INSERT INTO prescription VALUES (617, 411, 510, '1-0-0', 5, 5);

1 row created.

SQL> INSERT INTO prescription VALUES (618, 413, 506, '1-0-0', 10, 10);

1 row created.

SQL> INSERT INTO prescription VALUES (619, 413, 507, '0-0-1', 10, 10);

1 row created.

SQL> INSERT INTO prescription VALUES (620, 414, 506, '1-0-0', 10, 10);

1 row created.

SQL> INSERT INTO prescription VALUES (621, 414, 505, '1-0-0', 10, 10);

1 row created.

SQL> INSERT INTO prescription VALUES (622, 415, 502, '1-0-1', 5, 10);

1 row created.
```

Figure 24: *INSERT statements for PRESCRIPTION (613–622)*

```
SQL> PROMPT >>> BILL (11 rows, one per completed visit)
>>> BILL (11 rows, one per completed visit)
SQL> INSERT INTO bill VALUES (701, 401, 550, 21, 0, 571, 'Cash', 'Paid',
  2    DATE '2026-09-20');

1 row created.

SQL> INSERT INTO bill VALUES (702, 402, 500, 75, 0, 575, 'UPI', 'Paid',
  2    DATE '2026-09-22');

1 row created.

SQL> INSERT INTO bill VALUES (703, 403, 500, 78, 0, 578, 'Card', 'Paid',
  2    DATE '2026-09-25');

1 row created.

SQL> INSERT INTO bill VALUES (704, 404, 700, 45, 0, 745, 'UPI', 'Paid',
  2    DATE '2026-09-28');

1 row created.

SQL> INSERT INTO bill VALUES (705, 405, 500, 35, 0, 535, 'Cash', 'Paid',
  2    DATE '2026-10-01');

1 row created.

SQL> INSERT INTO bill VALUES (706, 406, 800, 315, 0, 1115, NULL, 'Pending',
  2    DATE '2026-10-01');

1 row created.

SQL> INSERT INTO bill VALUES (707, 407, 1000, 161, 15000, 16161, 'Card', 'Paid',
  2    DATE '2026-10-05');

1 row created.

SQL> INSERT INTO bill VALUES (708, 408, 700, 97.5, 0, 797.5, NULL, 'Pending',
  2    DATE '2026-10-03');

1 row created.

SQL> INSERT INTO bill VALUES (709, 409, 600, 18, 0, 618, 'Cash', 'Paid',
  2    DATE '2026-10-04');

1 row created.
```

Figure 25: *INSERT statements for BILL (701–707) – paid and pending bills*

```
SQL> INSERT INTO bill VALUES (710, 410, 1000, 273, 3000, 4273, NULL, 'Pending',
  2    DATE '2026-10-08');

1 row created.

SQL> INSERT INTO bill VALUES (711, 411, 900, 175, 0, 1075, NULL, 'Pending',
  2    DATE '2026-10-07');

1 row created.
```

Figure 26: *INSERT statements for BILL (710–711)*

```
SQL> PROMPT >>> AUDIT_LOG (12 historical entries)
>>> AUDIT_LOG (12 historical entries)
SQL> INSERT INTO audit_log VALUES (1, 'PATIENT', 'INSERT', 111, 'RECEPTION1',
  2    TIMESTAMP '2026-09-14 09:12:05', 'New patient registered');

1 row created.

SQL> INSERT INTO audit_log VALUES (2, 'PATIENT', 'INSERT', 112, 'RECEPTION1',
  2    TIMESTAMP '2026-09-15 10:03:41', 'New patient registered');

1 row created.

SQL> INSERT INTO audit_log VALUES (3, 'VISIT', 'INSERT', 412, 'RECEPTION1',
  2    TIMESTAMP '2026-10-05 12:20:10', 'New Appointment visit for patient 108');

1 row created.

SQL> INSERT INTO audit_log VALUES (4, 'VISIT', 'UPDATE', 412, 'RECEPTION1',
  2    TIMESTAMP '2026-10-07 16:45:02', 'Status Scheduled -> Cancelled');

1 row created.

SQL> INSERT INTO audit_log VALUES (5, 'VISIT', 'INSERT', 413, 'RECEPTION1',
  2    TIMESTAMP '2026-10-08 09:55:33', 'New Appointment visit for patient 109');

1 row created.

SQL> INSERT INTO audit_log VALUES (6, 'VISIT', 'UPDATE', 413, 'DR_RKUMAR',
  2    TIMESTAMP '2026-10-09 12:00:17', 'Status Scheduled -> Admitted');

1 row created.

SQL> INSERT INTO audit_log VALUES (7, 'VISIT', 'INSERT', 414, 'TRIAGE_NURSE',
  2    TIMESTAMP '2026-10-10 02:30:09', 'New Emergency visit for patient 110');

1 row created.

SQL> INSERT INTO audit_log VALUES (8, 'VISIT', 'UPDATE', 414, 'DR_AMENON',
  2    TIMESTAMP '2026-10-10 03:00:48', 'Status Waiting -> Admitted');

1 row created.

SQL> INSERT INTO audit_log VALUES (9, 'VISIT', 'INSERT', 415, 'TRIAGE_NURSE',
  2    TIMESTAMP '2026-10-12 20:10:21', 'New Emergency visit for patient 111');

1 row created.
```

Figure 27: *INSERT statements for AUDIT_LOG – historical entries 1–5*

```
SQL> INSERT INTO audit_log VALUES (10, 'VISIT', 'UPDATE', 415, 'DR_VSINGH',
  2    TIMESTAMP '2026-10-12 20:18:50', 'Status Waiting -> Under Treatment');

1 row created.

SQL> INSERT INTO audit_log VALUES (11, 'BILL', 'INSERT', 710, 'BILLING1',
  2    TIMESTAMP '2026-10-08 12:30:12', 'Bill generated: total 4273 (Pending)');

1 row created.

SQL> INSERT INTO audit_log VALUES (12, 'BILL', 'INSERT', 711, 'BILLING1',
  2    TIMESTAMP '2026-10-07 11:40:55', 'Bill generated: total 1075 (Pending)');

1 row created.
```

Figure 28: *INSERT statements for AUDIT_LOG – historical entries 10–12*

2.2 Commit and Row-Count Verification

```
SQL> PROMPT >>> AUDIT_LOG (12 historical entries)
>>> AUDIT_LOG (12 historical entries)
SQL> COMMIT;

Commit complete.

SQL>
SQL> PROMPT >>> Row-count check
>>> Row-count check
SQL> SELECT 'PATIENT' AS table_name, COUNT(*) AS row_count FROM patient
  2  UNION ALL
  3  SELECT 'DOCTOR' AS table_name, COUNT(*) AS row_count FROM doctor
  4  UNION ALL
  5  SELECT 'BED' AS table_name, COUNT(*) AS row_count FROM bed
  6  UNION ALL
  7  SELECT 'MEDICINE' AS table_name, COUNT(*) AS row_count FROM medicine
  8  UNION ALL
  9  SELECT 'VISIT' AS table_name, COUNT(*) AS row_count FROM visit
 10  UNION ALL
 11  SELECT 'PRESCRIPTION' AS table_name, COUNT(*) AS row_count FROM prescription
 12  UNION ALL
 13  SELECT 'BILL' AS table_name, COUNT(*) AS row_count FROM bill
 14  UNION ALL
 15  SELECT 'AUDIT_LOG' AS table_name, COUNT(*) AS row_count FROM audit_log;

TABLE_NAME    ROW_COUNT
------------ ----------
PATIENT              12
DOCTOR               10
BED                  12
MEDICINE             12
VISIT                20
PRESCRIPTION         22
BILL                 11
AUDIT_LOG            12

8 rows selected.
```

Figure 29: *COMMIT and row counts of all eight tables*

2.3 Constraint Validation

```
SQL>
SQL> PROMPT >>> Duplicate phone number (UNIQUE)
>>> Duplicate phone number (UNIQUE)
SQL> INSERT INTO patient (patient_id, patient_name, date_of_birth, phone)
  2  VALUES (150, 'Test Patient', DATE '2000-01-01', '9876500101');
INSERT INTO patient (patient_id, patient_name, date_of_birth, phone)
*
ERROR at line 1:
ORA-00001: unique constraint (HMS.UQ_PAT_PHONE) violated


SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-00001: unique constraint (HMS.UQ_PAT_PHONE) violated
[EXPECTED]   ORA-00001: unique constraint (HMS.UQ_PAT_PHONE) violated
SQL>
SQL> PROMPT >>> Triage level on a normal appointment (CHECK)
>>> Triage level on a normal appointment (CHECK)
SQL> INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type, visit_datetime,
  2                     triage_level)
  3  VALUES (151, 101, 202, 'Appointment', TIMESTAMP '2026-11-01 10:00:00', 2);
INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type, visit_datetime,
*
ERROR at line 1:
ORA-02290: check constraint (HMS.CHK_VISIT_TRIAGE) violated


SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-02290: check constraint (HMS.CHK_VISIT_TRIAGE) violated
[EXPECTED]   ORA-02290: check constraint (HMS.CHK_VISIT_TRIAGE) violated
SQL>
SQL> PROMPT >>> Doctor booked twice at the same time (unique index)
>>> Doctor booked twice at the same time (unique index)
SQL> INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type, visit_datetime)
  2  VALUES (152, 102, 210, 'Appointment', TIMESTAMP '2026-09-20 10:30:00');
INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type, visit_datetime)
*
ERROR at line 1:
ORA-00001: unique constraint (HMS.UQ_DOCTOR_SLOT) violated
```

Figure 30: *Constraint tests: duplicate phone, triage on an appointment, and doctor double-booking*

```
SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-00001: unique constraint (HMS.UQ_DOCTOR_SLOT) violated
[EXPECTED]   ORA-00001: unique constraint (HMS.UQ_DOCTOR_SLOT) violated
SQL>
SQL> PROMPT >>> Bed given to a second admitted patient (unique index)
>>> Bed given to a second admitted patient (unique index)
SQL> UPDATE visit SET bed_id = 305, visit_status = 'Admitted'
  2   WHERE visit_id = 414;
UPDATE visit SET bed_id = 305, visit_status = 'Admitted'
*
ERROR at line 1:
ORA-00001: unique constraint (HMS.UQ_ACTIVE_BED) violated


SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-00001: unique constraint (HMS.UQ_ACTIVE_BED) violated
[EXPECTED]   ORA-00001: unique constraint (HMS.UQ_ACTIVE_BED) violated
SQL>
SQL> PROMPT >>> Discharge time earlier than admit time (CHECK)
>>> Discharge time earlier than admit time (CHECK)
SQL> UPDATE visit SET discharge_time = TIMESTAMP '2026-10-01 00:00:00'
  2   WHERE visit_id = 407;
UPDATE visit SET discharge_time = TIMESTAMP '2026-10-01 00:00:00'
*
ERROR at line 1:
ORA-02290: check constraint (HMS.CHK_VISIT_DISCH) violated


SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-02290: check constraint (HMS.CHK_VISIT_DISCH) violated
[EXPECTED]   ORA-02290: check constraint (HMS.CHK_VISIT_DISCH) violated
SQL>
SQL> PROMPT >>> Medicine with zero price (CHECK)
>>> Medicine with zero price (CHECK)
SQL> INSERT INTO medicine VALUES (599, 'Test Tablet', 'ABC', 0, 10);
INSERT INTO medicine VALUES (599, 'Test Tablet', 'ABC', 0, 10)
*
ERROR at line 1:
ORA-02290: check constraint (HMS.CHK_MED_PRICE) violated
```

Figure 31: *Constraint tests: double-booked doctor, bed already occupied, and discharge earlier than admission*

```
SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-02290: check constraint (HMS.CHK_MED_PRICE) violated
[EXPECTED]   ORA-02290: check constraint (HMS.CHK_MED_PRICE) violated
SQL>
SQL> PROMPT >>> Bill marked Paid without a payment mode (CHECK)
>>> Bill marked Paid without a payment mode (CHECK)
SQL> UPDATE bill SET payment_status = 'Paid' WHERE bill_id = 706;
UPDATE bill SET payment_status = 'Paid' WHERE bill_id = 706
*
ERROR at line 1:
ORA-02290: check constraint (HMS.CHK_BILL_PAID) violated


SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-02290: check constraint (HMS.CHK_BILL_PAID) violated
[EXPECTED]   ORA-02290: check constraint (HMS.CHK_BILL_PAID) violated
SQL>
```

Figure 32: *Constraint tests: zero medicine price and a Paid bill without a payment mode*

2.4 INSERT, UPDATE and DELETE Operations

```
SQL> PROMPT >>> INSERT - register a new patient and book an appointment (keys come from sequences; registration_date takes its DEFAULT value)
>>> INSERT - register a new patient and book an appointment (keys come from sequences; registration_date takes its DEFAULT value)
SQL> INSERT INTO patient (patient_id, patient_name, gender, date_of_birth, phone,
  2                       address, blood_group, emergency_contact)
  3  VALUES (seq_patient.NEXTVAL, 'Sanjay Rao', 'Male', DATE '1992-08-17', '9876500113',
  4          '27 Beach Road, Visakhapatnam', 'O+', '9876511113');

1 row created.
```

Figure 33: *INSERT – register a new patient using seq_patient*

```
SQL>
SQL> INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type, visit_datetime,
  2                     complaint, visit_status)
  3  VALUES (seq_visit.NEXTVAL, 113, 202, 'Appointment',
  4          TIMESTAMP '2026-10-16 12:00:00', 'Acidity and stomach pain', 'Scheduled');

1 row created.
```

Figure 34: *INSERT – book an appointment using seq_visit*

```
SQL>
SQL> PROMPT >>> UPDATE - change a patient's contact details, revise fees, free a cleaned bed, restock a medicine and record a payment
>>> UPDATE - change a patient's contact details, revise fees, free a cleaned bed, restock a medicine and record a payment
SQL> UPDATE patient SET phone = '9876599105', address = '18 Park Street, Salt Lake, Kolkata'
  2   WHERE patient_id = 105;

1 row updated.
```

Figure 35: *UPDATE – change a patient's phone number and address*

```
SQL>
SQL> UPDATE doctor SET consultation_fee = consultation_fee * 1.10
  2   WHERE department = 'General Medicine';

2 rows updated.

SQL>
SQL> UPDATE bed SET bed_status = 'Free' WHERE bed_id = 304;

1 row updated.

SQL>
SQL> UPDATE medicine SET stock_quantity = stock_quantity + 200 WHERE medicine_id = 509;

1 row updated.

SQL>
SQL> UPDATE bill SET payment_status = 'Paid', payment_mode = 'UPI' WHERE bill_id = 710;

1 row updated.

SQL>
SQL> PROMPT >>> DELETE - remove a booking made by mistake together with the wrongly registered patient
>>> DELETE - remove a booking made by mistake together with the wrongly registered patient
SQL> UPDATE visit SET visit_status = 'Cancelled' WHERE visit_id = 421;

1 row updated.
```

Figure 36: *UPDATE – revise doctor fees, free bed 304, restock a medicine, record a payment and cancel visit 421*

```
SQL> DELETE FROM visit   WHERE visit_id = 421;

1 row deleted.

SQL> DELETE FROM patient WHERE patient_id = 113;

1 row deleted.

SQL> COMMIT;

Commit complete.
```

Figure 37: *DELETE – remove the mistaken visit and patient, then COMMIT*

```
SQL> PROMPT >>> Result of the changes
>>> Result of the changes
SQL> SELECT patient_id, phone, address FROM patient WHERE patient_id = 105;

PATIENT_ID PHONE           ADDRESS
---------- --------------- ----------------------------------------------------------------------------------------------------
       105 9876599105      18 Park Street, Salt Lake, Kolkata

1 row selected.

SQL>
SQL> PROMPT >>> Result of the changes
>>> Result of the changes
SQL> SELECT doctor_id, doctor_name, consultation_fee FROM doctor
  2   WHERE department = 'General Medicine' ORDER BY doctor_id;

 DOCTOR_ID DOCTOR_NAME                                          CONSULTATION_FEE
---------- -------------------------------------------------- ----------------
       202 Dr. Priya Nair                                                    550
       209 Dr. Suresh Babu                                                   550

2 rows selected.

SQL>
SQL> PROMPT >>> Result of the changes
>>> Result of the changes
SQL> SELECT bed_id, bed_status FROM bed WHERE bed_id = 304;

    BED_ID BED_STATUS
---------- ----------
       304 Free

1 row selected.

SQL>
SQL> PROMPT >>> Result of the changes
>>> Result of the changes
SQL> SELECT medicine_id, stock_quantity FROM medicine WHERE medicine_id = 509;

MEDICINE_ID STOCK_QUANTITY
----------- --------------
        509            208

1 row selected.
```

Figure 38: *Results of the updates: patient 105, General Medicine fees (now 550), bed 304 and medicine stock*

```
SQL>
SQL> PROMPT >>> Result of the changes
>>> Result of the changes
SQL> SELECT bill_id, payment_status, payment_mode FROM bill WHERE bill_id = 710;

   BILL_ID PAYMENT_ST PAYMENT_MO
---------- ---------- ----------
       710 Paid       UPI

1 row selected.

SQL>
SQL> PROMPT >>> Result of the changes
>>> Result of the changes
SQL> SELECT COUNT(*) AS patient_113 FROM patient WHERE patient_id = 113;

PATIENT_113
-----------
          0

1 row selected.

SQL>
SQL> PROMPT >>> A patient who has visits cannot be deleted, so history is never lost by accident.
>>> A patient who has visits cannot be deleted, so history is never lost by accident.
SQL> DELETE FROM patient WHERE patient_id = 101;
DELETE FROM patient WHERE patient_id = 101
*
ERROR at line 1:
ORA-02292: integrity constraint (HMS.FK_VISIT_PATIENT) violated - child record found


SQL> PROMPT [EXPECTED in report]
[EXPECTED in report]
SQL> PROMPT [EXPECTED]   ORA-02292: integrity constraint (HMS.FK_VISIT_PATIENT) violated - child record found
[EXPECTED]   ORA-02292: integrity constraint (HMS.FK_VISIT_PATIENT) violated - child record found
```

Figure 39: *Result of the payment update and deletion; deleting a patient with visits is blocked by the foreign key (ORA-02292)*

3. SQL Queries

```
SQL> PROMPT >>> Concept: JOIN of three tables (FR2)
>>> Concept: JOIN of three tables (FR2)
SQL> SELECT v.visit_id,
  2         TO_CHAR(v.visit_datetime, 'YYYY-MM-DD HH24:MI') AS visit_time,
  3         v.visit_type, d.doctor_name, v.complaint, v.visit_status
  4    FROM patient p
  5    JOIN visit  v ON v.patient_id = p.patient_id
  6    JOIN doctor d ON d.doctor_id  = v.doctor_id
  7   WHERE p.patient_name = 'Aakash Gupta'
  8   ORDER BY v.visit_datetime;

  VISIT_ID VISIT_TIME       VISIT_TYPE   DOCTOR_NAME                                        COMPLAINT
---------- ---------------- ------------ -------------------------------------------------- ----------------------------------------
VISIT_STATUS
---------------
       401 2026-09-20 10:30 Appointment  Dr. Anita Desai                                    Skin rash
Discharged

       411 2026-10-07 11:00 Appointment  Dr. Karthik Raman                                  Recurring migraine
Discharged


2 rows selected.

SQL>
SQL> PROMPT >>> Concept: LEFT JOIN, COUNT, GROUP BY, ORDER BY (FR20)
>>> Concept: LEFT JOIN, COUNT, GROUP BY, ORDER BY (FR20)
SQL> SELECT d.doctor_name, d.department, COUNT(v.visit_id) AS total_visits
  2    FROM doctor d
  3    LEFT JOIN visit v ON v.doctor_id = d.doctor_id
  4   GROUP BY d.doctor_name, d.department
  5   ORDER BY total_visits DESC, d.doctor_name;

DOCTOR_NAME                                        DEPARTMENT                    TOTAL_VISITS
-------------------------------------------------- ----------------------------- ------------
Dr. Arvind Menon                                   Emergency                                4
Dr. Priya Nair                                     General Medicine                         3
Dr. Rajesh Kumar                                   Cardiology                               3
Dr. Vikram Singh                                   Emergency                                3
Dr. Sneha Iyer                                     Orthopedics                              2
Dr. Anita Desai                                    Dermatology                              1
Dr. Fatima Sheikh                                  Pediatrics                               1
Dr. Karthik Raman                                  Neurology                                1
Dr. Meena Pillai                                   Gynecology                               1
Dr. Suresh Babu                                    General Medicine                         1

10 rows selected.
```

Figure 40: *Query A: a patient's visit history (three-table JOIN) and Query B: visits per doctor (LEFT JOIN with GROUP BY)*

```
SQL> PROMPT >>> Concept: GROUP BY with CASE (FR20)
>>> Concept: GROUP BY with CASE (FR20)
SQL> SELECT triage_level,
  2         CASE triage_level WHEN 1 THEN 'Critical' WHEN 2 THEN 'Very urgent'
  3                           WHEN 3 THEN 'Urgent'   ELSE 'Minor' END AS category,
  4         COUNT(*) AS cases
  5    FROM visit
  6   WHERE visit_type = 'Emergency'
  7   GROUP BY triage_level
  8   ORDER BY triage_level;

TRIAGE_LEVEL CATEGORY         CASES
------------ ---------- ----------
           1 Critical            3
           2 Very urgent         2
           3 Urgent              1
           4 Minor               1

4 rows selected.

SQL>
SQL> PROMPT >>> Concept: JOIN, WHERE, GROUP BY, HAVING
>>> Concept: JOIN, WHERE, GROUP BY, HAVING
SQL> SELECT d.department, COUNT(*) AS total_visits
  2    FROM doctor d
  3    JOIN visit v ON v.doctor_id = d.doctor_id
  4   WHERE v.visit_status <> 'Cancelled'
  5   GROUP BY d.department
  6  HAVING COUNT(*) >= 3
  7   ORDER BY total_visits DESC, d.department;

DEPARTMENT                    TOTAL_VISITS
----------------------------- ------------
Emergency                                7
General Medicine                         4
Cardiology                               3

3 rows selected.
```

Figure 41: *Query C: emergency cases by triage category (CASE) and Query D: departments with at least three visits (HAVING)*

```
SQL> PROMPT >>> Concept: Simple filter with ORDER BY (FR11)
>>> Concept: Simple filter with ORDER BY (FR11)
SQL> SELECT medicine_id, medicine_name, stock_quantity
  2    FROM medicine
  3   WHERE stock_quantity < 20
  4   ORDER BY stock_quantity;

MEDICINE_ID MEDICINE_NAME                                      STOCK_QUANTITY
----------- -------------------------------------------------- --------------
        512 ORS Sachet                                                      0
        510 Sumatriptan 50mg                                                5

2 rows selected.

SQL>
SQL> PROMPT >>> Concept: Aggregate functions on a filtered set (FR12)
>>> Concept: Aggregate functions on a filtered set (FR12)
SQL> SELECT ward_type, COUNT(*) AS free_beds, MIN(daily_charge) AS daily_charge
  2    FROM bed
  3   WHERE bed_status = 'Free'
  4   GROUP BY ward_type
  5   ORDER BY ward_type;

WARD_TYPE        FREE_BEDS DAILY_CHARGE
--------------- ---------- ------------
Emergency                3         1500
General                  4          600
ICU                      1         5000
Private                  1         2500

4 rows selected.
```

Figure 42: *Query E: medicines with stock below 20 and Query F: free beds per ward*

```
SQL> PROMPT >>> Concept: Conditional aggregation with SUM(CASE ...)
>>> Concept: Conditional aggregation with SUM(CASE ...)
SQL> SELECT ward_type,
  2         COUNT(*) AS total_beds,
  3         SUM(CASE WHEN bed_status = 'Occupied' THEN 1 ELSE 0 END) AS occupied,
  4         ROUND(100.0 * SUM(CASE WHEN bed_status = 'Occupied' THEN 1 ELSE 0 END)
  5               / COUNT(*), 1) AS occupancy_pct
  6    FROM bed
  7   GROUP BY ward_type
  8   ORDER BY ward_type;

WARD_TYPE       TOTAL_BEDS   OCCUPIED OCCUPANCY_PCT
--------------- ---------- ---------- -------------
Emergency                4          0             0
General                  4          0             0
ICU                      2          1            50
Private                  2          1            50

4 rows selected.

SQL>
SQL> PROMPT >>> Concept: SUM / COUNT grouped by payment status (FR18)
>>> Concept: SUM / COUNT grouped by payment status (FR18)
SQL> SELECT payment_status, COUNT(*) AS bills, SUM(total_amount) AS amount
  2    FROM bill
  3   GROUP BY payment_status
  4   ORDER BY payment_status;

PAYMENT_ST      BILLS     AMOUNT
---------- ---------- ----------
Paid                8      24056
Pending             3     2987.5

2 rows selected.
```

Figure 43: *Query G: bed occupancy percentage by ward and Query H: bills grouped by payment status*

```
SQL> PROMPT >>> Concept: Date formatting inside GROUP BY (FR20)
>>> Concept: Date formatting inside GROUP BY (FR20)
SQL> SELECT TO_CHAR(bill_date, 'YYYY-MM') AS bill_month,
  2         COUNT(*) AS bills,
  3         SUM(consultation_charge) AS consultation,
  4         SUM(medicine_charge)     AS medicine,
  5         SUM(bed_charge)          AS beds,
  6         SUM(total_amount)        AS total
  7    FROM bill
  8   GROUP BY TO_CHAR(bill_date, 'YYYY-MM')
  9   ORDER BY bill_month;

BILL_MO      BILLS CONSULTATION   MEDICINE       BEDS      TOTAL
------- ---------- ------------ ---------- ---------- ----------
2026-09          4         2250        219          0       2469
2026-10          7         5500     1074.5      18000    24574.5

2 rows selected.

SQL>
SQL> PROMPT >>> Concept: Four-table JOIN (FR18)
>>> Concept: Four-table JOIN (FR18)
SQL> SELECT b.bill_id, p.patient_name, d.doctor_name, b.total_amount
  2    FROM bill b
  3    JOIN visit   v ON v.visit_id   = b.visit_id
  4    JOIN patient p ON p.patient_id = v.patient_id
  5    JOIN doctor  d ON d.doctor_id  = v.doctor_id
  6   WHERE b.payment_status = 'Pending'
  7   ORDER BY b.total_amount DESC;

   BILL_ID PATIENT_NAME                                       DOCTOR_NAME                                        TOTAL_AMOUNT
---------- -------------------------------------------------- -------------------------------------------------- ------------
       706 Kavita Sharma                                      Dr. Rajesh Kumar                                           1115
       711 Aakash Gupta                                       Dr. Karthik Raman                                          1075
       708 Divya Krishnan                                     Dr. Sneha Iyer                                           797.5

3 rows selected.
```

Figure 44: *Query I: monthly billing summary and Query J: pending bills (four-table JOIN)*

```
SQL>
SQL> PROMPT >>> Concept: Correlated subquery with NOT EXISTS
>>> Concept: Correlated subquery with NOT EXISTS
SQL> SELECT p.patient_id, p.patient_name, p.blood_group
  2    FROM patient p
  3   WHERE NOT EXISTS (SELECT 1
  4                     FROM visit v
  5                    WHERE v.patient_id = p.patient_id
  6                      AND v.visit_type = 'Emergency')
  7   ORDER BY p.patient_id;

PATIENT_ID PATIENT_NAME                                       BLOOD
---------- -------------------------------------------------- -----
       101 Arjun Reddy                                        B+
       102 Kavita Sharma                                      O+
       105 Mohammed Irfan                                     O-
       107 Aakash Gupta                                       A-
       108 Lakshmi Narayanan                                  O+
       109 Vivek Joshi                                        B+

6 rows selected.

SQL>
SQL> PROMPT >>> Concept: Scalar subquery
>>> Concept: Scalar subquery
SQL> SELECT doctor_name, department, consultation_fee
  2    FROM doctor
  3   WHERE consultation_fee > (SELECT AVG(consultation_fee) FROM doctor)
  4   ORDER BY consultation_fee DESC, doctor_name;

DOCTOR_NAME                                        DEPARTMENT                     CONSULTATION_FEE
-------------------------------------------------- ------------------------------ ----------------
Dr. Arvind Menon                                   Emergency                              1000
Dr. Vikram Singh                                   Emergency                              1000
Dr. Karthik Raman                                  Neurology                               900
Dr. Rajesh Kumar                                   Cardiology                              800
Dr. Meena Pillai                                   Gynecology                              750

5 rows selected.
```

Figure 45: *Query K: patients with no emergency visit (NOT EXISTS) and Query L: doctors above the average fee (scalar subquery)*

```
SQL> PROMPT >>> Concept: Subquery with RANK() window function
>>> Concept: Subquery with RANK() window function
SQL> SELECT patient_name, total_billed, rnk
  2    FROM (SELECT p.patient_name,
  3                 SUM(b.total_amount) AS total_billed,
  4                 RANK() OVER (ORDER BY SUM(b.total_amount) DESC) AS rnk
  5            FROM bill b
  6            JOIN visit   v ON v.visit_id   = b.visit_id
  7            JOIN patient p ON p.patient_id = v.patient_id
  8           GROUP BY p.patient_id, p.patient_name) t
  9   WHERE rnk <= 3
 10   ORDER BY rnk, patient_name;

PATIENT_NAME                                       TOTAL_BILLED        RNK
-------------------------------------------------- ------------ ----------
Rahul Verma                                           16161          1
Sunita Patil                                           4273          2
Aakash Gupta                                           1646          3

3 rows selected.

SQL>
SQL> PROMPT >>> Concept: JOIN, aggregate over a computed value, HAVING
>>> Concept: JOIN, aggregate over a computed value, HAVING
SQL> SELECT m.medicine_name,
  2         COUNT(*)                      AS times_prescribed,
  3         SUM(pr.quantity)              AS units,
  4         SUM(pr.quantity * m.price_per_unit) AS total_value
  5    FROM prescription pr
  6    JOIN medicine m ON m.medicine_id = pr.medicine_id
  7   GROUP BY m.medicine_name
  8  HAVING COUNT(*) > 1
  9   ORDER BY units DESC, m.medicine_name;

MEDICINE_NAME                                      TIMES_PRESCRIBED      UNITS TOTAL_VALUE
-------------------------------------------------- ---------------- ---------- -----------
Aspirin 75mg                                                      3         50          75
Atorvastatin 10mg                                                 2         40         360
Ibuprofen 400mg                                                   3         39       175.5
Paracetamol 500mg                                                 3         25          50
Amoxicillin 500mg                                                 2         24         204
Pantoprazole 40mg                                                 3         22         132
Cetirizine 10mg                                                   2         12          36

7 rows selected.
```

Figure 46: *Query M: top three billed patients using RANK() and Query N: prescription statistics per medicine*

```
SQL> PROMPT >>> Concept: Multi-table JOIN with status filter
>>> Concept: Multi-table JOIN with status filter
SQL> SELECT v.visit_id, p.patient_name, v.visit_type, b.bed_id, b.ward_type,
  2         b.daily_charge, d.doctor_name
  3    FROM visit v
  4    JOIN patient p ON p.patient_id = v.patient_id
  5    JOIN bed     b ON b.bed_id     = v.bed_id
  6    JOIN doctor  d ON d.doctor_id  = v.doctor_id
  7   WHERE v.visit_status = 'Admitted'
  8   ORDER BY b.ward_type, v.visit_id;

  VISIT_ID PATIENT_NAME                                       VISIT_TYPE       BED_ID WARD_TYPE       DAILY_CHARGE DOCTOR_NAME
---------- -------------------------------------------------- ------------ ---------- -------------- ------------ ----------------------------------
       414 Neha Kapoor                                        Emergency           308 ICU                    5000 Dr. Arvind Menon
       413 Vivek Joshi                                        Appointment         305 Private                2500 Dr. Rajesh Kumar

2 rows selected.

SQL>
SQL> PROMPT >>> Concept: ORDER BY on two columns (FR7)
>>> Concept: ORDER BY on two columns (FR7)
SQL> SELECT v.visit_id, p.patient_name, v.triage_level,
  2         TO_CHAR(v.visit_datetime, 'HH24:MI') AS arrived, v.complaint
  3    FROM visit v
  4    JOIN patient p ON p.patient_id = v.patient_id
  5   WHERE v.visit_type = 'Emergency' AND v.visit_status = 'Waiting'
  6   ORDER BY v.triage_level, v.visit_datetime;

  VISIT_ID PATIENT_NAME                                       TRIAGE_LEVEL ARRIV COMPLAINT
---------- -------------------------------------------------- ------------ ----- --------------------------------------------------------------------------------
       418 Divya Krishnan                                                1 20:50 Severe allergic reaction
       416 Pooja Menon                                                   2 20:25 High fever and vomiting
       417 Rahul Verma                                                   4 20:40 Minor ankle sprain

3 rows selected.
```

Figure 47: *Query O: admitted patients with ward and bed charge, and Query P: emergency queue ordered by triage then arrival*

```
SQL> PROMPT >>> Concept: JOIN with AVG and ROUND
>>> Concept: JOIN with AVG and ROUND
SQL> SELECT v.visit_type, COUNT(*) AS bills,
  2         ROUND(AVG(b.total_amount), 2) AS avg_bill,
  3         MAX(b.total_amount)           AS highest_bill
  4    FROM bill b
  5    JOIN visit v ON v.visit_id = b.visit_id
  6   GROUP BY v.visit_type
  7   ORDER BY v.visit_type;

VISIT_TYPE        BILLS   AVG_BILL HIGHEST_BILL
------------ ---------- ---------- ------------
Appointment           9     734.39         1115
Emergency             2      10217        16161

2 rows selected.

SQL>
SQL> PROMPT >>> Concept: Nested subquery with IN
>>> Concept: Nested subquery with IN
SQL> SELECT doctor_id, doctor_name, department
  2    FROM doctor
  3   WHERE doctor_id IN (SELECT v.doctor_id
  4                         FROM visit v
  5                         JOIN prescription pr ON pr.visit_id = v.visit_id
  6                         JOIN medicine m ON m.medicine_id = pr.medicine_id
  7                        WHERE m.medicine_name LIKE 'Aspirin%')
  8   ORDER BY doctor_id;

 DOCTOR_ID DOCTOR_NAME                                         DEPARTMENT
---------- -------------------------------------------------- ------------------------------
       201 Dr. Rajesh Kumar                                   Cardiology
       203 Dr. Arvind Menon                                   Emergency

2 rows selected.
```

Figure 48: *Query Q: average and highest bill by visit type and Query R: doctors who prescribed Aspirin (IN subquery)*

```
SQL>
SQL> PROMPT >>> Concept: GROUP BY, HAVING, MIN / MAX dates
>>> Concept: GROUP BY, HAVING, MIN / MAX dates
SQL> SELECT p.patient_name,
  2         COUNT(*) AS visits,
  3         TO_CHAR(MIN(v.visit_datetime), 'YYYY-MM-DD') AS first_visit,
  4         TO_CHAR(MAX(v.visit_datetime), 'YYYY-MM-DD') AS last_visit
  5    FROM patient p
  6    JOIN visit v ON v.patient_id = p.patient_id
  7   GROUP BY p.patient_id, p.patient_name
  8  HAVING COUNT(*) > 1
  9   ORDER BY visits DESC, p.patient_name;

PATIENT_NAME                                        VISITS FIRST_VISI LAST_VISIT
-------------------------------------------------- ---------- ---------- ----------
Aakash Gupta                                                2 2026-09-20 2026-10-07
Divya Krishnan                                              2 2026-10-03 2026-10-12
Kavita Sharma                                               2 2026-10-01 2026-10-15
Lakshmi Narayanan                                           2 2026-09-22 2026-10-08
Mohammed Irfan                                              2 2026-10-04 2026-10-14
Neha Kapoor                                                 2 2026-09-25 2026-10-10
Rahul Verma                                                 2 2026-10-02 2026-10-12
Vivek Joshi                                                 2 2026-09-28 2026-10-09

8 rows selected.
```

Figure 49: *Query S: patients with more than one visit, with first and last visit dates*

4. Procedures and Functions

4.1 Functions

Function	Description
fn_days_stayed(visit_id)	Number of bed days charged: 0 if never admitted, otherwise at least one day; uses the current time if the patient has not been discharged.
fn_medicine_cost(visit_id)	Total value of prescribed medicines (quantity × unit price); returns 0 when nothing is prescribed.
fn_triage_label(level)	Converts triage level 1–4 into Critical, Very urgent, Urgent or Minor (DETERMINISTIC).

```
SQL> CREATE OR REPLACE FUNCTION fn_days_stayed (p_visit_id IN NUMBER)
  2  RETURN NUMBER
  3  IS
  4  v_admit  visit.admit_time%TYPE;
  5  v_disch  visit.discharge_time%TYPE;
  6  v_days   NUMBER;
  7  BEGIN
  8  SELECT admit_time, discharge_time
  9  INTO v_admit, v_disch
 10  FROM visit
 11  WHERE visit_id = p_visit_id;
 12
 13  IF v_admit IS NULL THEN
 14  RETURN 0;                                   -- never admitted: no bed days
 15  END IF;
 16
 17  v_days := TRUNC(CAST(NVL(v_disch, LOCALTIMESTAMP) AS DATE))
 18  - TRUNC(CAST(v_admit AS DATE));
 19  RETURN GREATEST(v_days, 1);                  -- at least one day is charged
 20  EXCEPTION
 21  WHEN NO_DATA_FOUND THEN
 22  RAISE_APPLICATION_ERROR(-20001, 'Visit ' || p_visit_id || ' does not exist');
 23  END fn_days_stayed;
 24  /

Function created.
```

Figure 50: *Function fn_days_stayed*

```
SQL> CREATE OR REPLACE FUNCTION fn_medicine_cost (p_visit_id IN NUMBER)
  2  RETURN NUMBER
  3  IS
  4  v_total  NUMBER;
  5  BEGIN
  6  SELECT NVL(SUM(pr.quantity * m.price_per_unit), 0)
  7  INTO v_total
  8  FROM prescription pr
  9  JOIN medicine m ON m.medicine_id = pr.medicine_id
 10  WHERE pr.visit_id = p_visit_id;
 11
 12  RETURN v_total;
 13  END fn_medicine_cost;
 14  /

Function created.
```

Figure 51: *Function fn_medicine_cost*

```
SQL> CREATE OR REPLACE FUNCTION fn_triage_label (p_level IN NUMBER)
  2  RETURN VARCHAR2
  3  DETERMINISTIC
  4  IS
  5  BEGIN
  6  RETURN CASE p_level
  7  WHEN 1 THEN 'Critical'
  8  WHEN 2 THEN 'Very urgent'
  9  WHEN 3 THEN 'Urgent'
 10  WHEN 4 THEN 'Minor'
 11  ELSE 'Not triaged'
 12  END;
 13  END fn_triage_label;
 14  /

Function created.
```

Figure 52: *Function fn_triage_label*

```
SQL> SELECT visit_id,
  2  fn_days_stayed(visit_id)   AS days_stayed,
  3  fn_medicine_cost(visit_id) AS medicine_cost,
  4  fn_triage_label(triage_level) AS triage
  5  FROM visit
  6  WHERE visit_id IN (405, 407, 410)
  7  ORDER BY visit_id;

  VISIT_ID DAYS_STAYED MEDICINE_COST
---------- ----------- -------------
TRIAGE
--------------------------------------------------------------------------------------------------------------------------------------------------------
--------------------------------------------
       405           0            35
Not triaged

       407           3           161
Critical

       410           2           273
Very urgent
```

Figure 53: *Testing the three functions on visits 405, 407 and 410 in a single query*

4.2 Procedures

Procedure	Description
sp_book_appointment	Checks that the patient and doctor exist, the slot is in the future and the doctor is free, then inserts a Scheduled visit and returns its id. A DUP_VAL_ON_INDEX handler protects against two users booking the same slot simultaneously.

Procedure	Description
sp_register_emergency	Validates the triage level, then automatically assigns the Emergency doctor with the fewest active cases (ties go to the lowest id) and creates a Waiting visit.
sp_allot_bed	Locks the visit, rejects cancelled, discharged or already-bedded visits, picks the lowest-numbered free bed in the requested ward, marks it Occupied and admits the patient.
sp_discharge_patient	Discharges a Waiting, Under Treatment or Admitted visit, records the discharge time for admitted patients and sets their bed to Cleaning.

```
SQL> CREATE OR REPLACE PROCEDURE sp_book_appointment (
  2  p_patient_id  IN  NUMBER,
  3  p_doctor_id   IN  NUMBER,
  4  p_slot        IN  TIMESTAMP,
  5  p_complaint   IN  VARCHAR2,
  6  p_visit_id    OUT NUMBER
  7  )
  8  IS
  9  v_count  NUMBER;
 10  BEGIN
 11  SELECT COUNT(*) INTO v_count FROM patient WHERE patient_id = p_patient_id;
 12  IF v_count = 0 THEN
 13  RAISE_APPLICATION_ERROR(-20002, 'Patient ' || p_patient_id || ' is not registered');
 14  END IF;
 15
 16  SELECT COUNT(*) INTO v_count FROM doctor WHERE doctor_id = p_doctor_id;
 17  IF v_count = 0 THEN
 18  RAISE_APPLICATION_ERROR(-20003, 'Doctor ' || p_doctor_id || ' does not exist');
 19  END IF;
 20
 21  IF p_slot <= LOCALTIMESTAMP THEN
 22  RAISE_APPLICATION_ERROR(-20004, 'Appointment time must be in the future');
 23  END IF;
 24
 25  SELECT COUNT(*) INTO v_count
 26  FROM visit
 27  WHERE doctor_id = p_doctor_id
 28  AND visit_datetime = p_slot
 29  AND visit_status <> 'Cancelled';
 30  IF v_count > 0 THEN
 31  RAISE_APPLICATION_ERROR(-20005,
 32  'Doctor ' || p_doctor_id || ' already has an appointment at that time');
 33  END IF;
 34
```

Figure 54: *Procedure sp_book_appointment – input validation (part 1)*

```
 35  INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type,
 36  visit_datetime, complaint, visit_status)
 37  VALUES (seq_visit.NEXTVAL, p_patient_id, p_doctor_id, 'Appointment',
 38  p_slot, p_complaint, 'Scheduled')
 39  RETURNING visit_id INTO p_visit_id;
 40
 41  COMMIT;
 42  EXCEPTION
 43  WHEN DUP_VAL_ON_INDEX THEN                    -- two users booking at once
 44  ROLLBACK;
 45  RAISE_APPLICATION_ERROR(-20005,
 46  'Doctor ' || p_doctor_id || ' already has an appointment at that time');
 47  END sp_book_appointment;
 48  /

Procedure created.
```

Figure 55: *Procedure sp_book_appointment – insert and concurrency handling (part 2)*

```
SQL> CREATE OR REPLACE PROCEDURE sp_register_emergency (
  2  p_patient_id  IN  NUMBER,
  3  p_complaint   IN  VARCHAR2,
  4  p_triage      IN  NUMBER,
  5  p_visit_id    OUT NUMBER,
  6  p_arrival     IN  TIMESTAMP DEFAULT LOCALTIMESTAMP
  7  )
  8  IS
  9  v_count   NUMBER;
 10  v_doctor  doctor.doctor_id%TYPE;
 11  BEGIN
 12  IF p_triage NOT BETWEEN 1 AND 4 THEN
 13  RAISE_APPLICATION_ERROR(-20010, 'Triage level must be between 1 and 4');
 14  END IF;
 15
 16  SELECT COUNT(*) INTO v_count FROM patient WHERE patient_id = p_patient_id;
 17  IF v_count = 0 THEN
 18  RAISE_APPLICATION_ERROR(-20002, 'Patient ' || p_patient_id || ' is not registered');
 19  END IF;
 20
 21  -- emergency doctor with the smallest number of active cases (ties: lowest id)
 22  BEGIN
 23  SELECT doctor_id
 24  INTO v_doctor
 25  FROM (SELECT d.doctor_id
 26  FROM doctor d
 27  WHERE d.department = 'Emergency'
 28  ORDER BY (SELECT COUNT(*)
 29  FROM visit v
 30  WHERE v.doctor_id = d.doctor_id
 31  AND v.visit_status IN ('Waiting', 'Under Treatment')),
 32  d.doctor_id)
 33  WHERE ROWNUM = 1;
 34  EXCEPTION
 35  WHEN NO_DATA_FOUND THEN
 36  RAISE_APPLICATION_ERROR(-20011, 'No emergency doctor is available');
 37  END;
```

Figure 56: *Procedure sp_register_emergency – triage validation and automatic doctor selection (part 1)*

```
 39  INSERT INTO visit (visit_id, patient_id, doctor_id, visit_type, visit_datetime,
 40  complaint, triage_level, visit_status)
 41  VALUES (seq_visit.NEXTVAL, p_patient_id, v_doctor, 'Emergency', p_arrival,
 42  p_complaint, p_triage, 'Waiting')
 43  RETURNING visit_id INTO p_visit_id;
 44
 45  COMMIT;
 46  END sp_register_emergency;
 47  /

Procedure created.
```

Figure 57: *Procedure sp_register_emergency – creation of the emergency visit (part 2)*

```
SQL> CREATE OR REPLACE PROCEDURE sp_allot_bed (
  2  p_visit_id   IN  NUMBER,
  3  p_ward_type  IN  VARCHAR2,
  4  p_bed_id     OUT NUMBER
  5  )
  6  IS
  7  v_status  visit.visit_status%TYPE;
  8  v_bed     visit.bed_id%TYPE;
  9  BEGIN
 10  BEGIN
 11  SELECT visit_status, bed_id
 12  INTO v_status, v_bed
 13  FROM visit
 14  WHERE visit_id = p_visit_id
 15  FOR UPDATE;
 16  EXCEPTION
 17  WHEN NO_DATA_FOUND THEN
 18  RAISE_APPLICATION_ERROR(-20020, 'Visit ' || p_visit_id || ' does not exist');
 19  END;
 20
 21  IF v_status IN ('Cancelled', 'Discharged') THEN
 22  RAISE_APPLICATION_ERROR(-20021, 'Visit ' || p_visit_id || ' is already ' || v_status);
 23  END IF;
 24  IF v_bed IS NOT NULL THEN
 25  RAISE_APPLICATION_ERROR(-20022, 'Visit ' || p_visit_id || ' already has a bed');
 26  END IF;
 27
```

Figure 58: *Procedure sp_allot_bed – row locking and visit checks (part 1)*

```
 28  BEGIN
 29  SELECT bed_id
 30  INTO p_bed_id
 31  FROM bed
 32  WHERE bed_id = (SELECT MIN(bed_id)
 33  FROM bed
 34  WHERE ward_type = p_ward_type
 35  AND bed_status = 'Free')
 36  FOR UPDATE;
 37  EXCEPTION
 38  WHEN NO_DATA_FOUND THEN
 39  RAISE_APPLICATION_ERROR(-20023, 'No free bed in the ' || p_ward_type || ' ward');
 40  END;
 41
 42  UPDATE bed
 43  SET bed_status = 'Occupied'
 44  WHERE bed_id = p_bed_id AND bed_status = 'Free';
 45  IF SQL%ROWCOUNT = 0 THEN
 46  RAISE_APPLICATION_ERROR(-20024, 'Bed ' || p_bed_id || ' was taken, try again');
 47  END IF;
 48
 49  UPDATE visit
 50  SET bed_id = p_bed_id,
 51  admit_time = LOCALTIMESTAMP,
 52  visit_status = 'Admitted'
 53  WHERE visit_id = p_visit_id;
 54
 55  COMMIT;
 56  END sp_allot_bed;
 57  /

Procedure created.
```

Figure 59: *Procedure sp_allot_bed – bed selection, occupancy and admission (part 2)*

```
SQL> CREATE OR REPLACE PROCEDURE sp_discharge_patient (p_visit_id IN NUMBER)
  2  IS
  3  v_status  visit.visit_status%TYPE;
  4  v_bed     visit.bed_id%TYPE;
  5  BEGIN
  6  BEGIN
  7  SELECT visit_status, bed_id
  8  INTO v_status, v_bed
  9  FROM visit
 10  WHERE visit_id = p_visit_id
 11  FOR UPDATE;
 12  EXCEPTION
 13  WHEN NO_DATA_FOUND THEN
 14  RAISE_APPLICATION_ERROR(-20030, 'Visit ' || p_visit_id || ' does not exist');
 15  END;
 16
 17  IF v_status NOT IN ('Waiting', 'Under Treatment', 'Admitted') THEN
 18  RAISE_APPLICATION_ERROR(-20031,
 19  'Visit ' || p_visit_id || ' cannot be discharged from status ' || v_status);
 20  END IF;
 21
 22  UPDATE visit
 23  SET visit_status   = 'Discharged',
 24  discharge_time = CASE WHEN bed_id IS NOT NULL THEN LOCALTIMESTAMP END
 25  WHERE visit_id = p_visit_id;
 26
 27  IF v_bed IS NOT NULL THEN
 28  UPDATE bed SET bed_status = 'Cleaning' WHERE bed_id = v_bed;
 29  END IF;
 30
 31  COMMIT;
 32  END sp_discharge_patient;
 33  /

Procedure created.
```

Figure 60: *Procedure sp_discharge_patient*

5. Cursors and Triggers

5.1 Cursors

```
SQL> CREATE OR REPLACE PROCEDURE sp_emergency_queue
  2  IS
  3  CURSOR c_queue IS
  4  SELECT v.visit_id, p.patient_name, v.triage_level, v.complaint,
  5  d.doctor_name, v.visit_datetime
  6  FROM visit v
  7  JOIN patient p ON p.patient_id = v.patient_id
  8  JOIN doctor  d ON d.doctor_id  = v.doctor_id
  9  WHERE v.visit_type   = 'Emergency'
 10  AND v.visit_status = 'Waiting'
 11  ORDER BY v.triage_level, v.visit_datetime;
 12
 13  r  c_queue%ROWTYPE;
 14  BEGIN
 15  DBMS_OUTPUT.PUT_LINE('EMERGENCY WAITING LIST (highest priority first)');
 16  DBMS_OUTPUT.PUT_LINE(RPAD('-', 70, '-'));
 17
 18  OPEN c_queue;
 19  LOOP
 20  FETCH c_queue INTO r;
 21  EXIT WHEN c_queue%NOTFOUND;
 22  DBMS_OUTPUT.PUT_LINE(c_queue%ROWCOUNT || '. [T' || r.triage_level || '] '
 23  || RPAD(fn_triage_label(r.triage_level), 11)
 24  || '| Visit ' || r.visit_id || ' | ' || r.patient_name
 25  || ' | arrived ' || TO_CHAR(r.visit_datetime, 'HH24:MI'));
 26  DBMS_OUTPUT.PUT_LINE('      Complaint: ' || r.complaint
 27  || ' | ' || r.doctor_name);
 28  END LOOP;
 29
 30  IF c_queue%ROWCOUNT = 0 THEN
 31  DBMS_OUTPUT.PUT_LINE('No emergency patient is waiting.');
 32  END IF;
 33  CLOSE c_queue;
 34  END sp_emergency_queue;
 35  /

Procedure created.
```

Figure 61: *Procedure sp_emergency_queue – explicit cursor over waiting emergency patients*

```
SQL> CREATE OR REPLACE PROCEDURE sp_pending_bills (p_min_amount IN NUMBER DEFAULT 0)
  2  IS
  3  CURSOR c_pending (cp_min NUMBER) IS
  4  SELECT b.bill_id, p.patient_name, b.bill_date, b.total_amount
  5  FROM bill b
  6  JOIN visit   v ON v.visit_id   = b.visit_id
  7  JOIN patient p ON p.patient_id = v.patient_id
  8  WHERE b.payment_status = 'Pending'
  9  AND b.total_amount  >= cp_min
 10  ORDER BY b.bill_date, b.bill_id;
 11
 12  v_count  NUMBER := 0;
 13  v_sum    NUMBER := 0;
 14  BEGIN
 15  DBMS_OUTPUT.PUT_LINE('PENDING BILLS (amount >= ' || p_min_amount || ')');
 16  DBMS_OUTPUT.PUT_LINE(RPAD('-', 62, '-'));
 17
 18  FOR r IN c_pending(p_min_amount) LOOP
 19  DBMS_OUTPUT.PUT_LINE('Bill ' || r.bill_id || ' | ' || RPAD(r.patient_name, 18)
 20  || ' | ' || TO_CHAR(r.bill_date, 'DD-Mon-YYYY')
 21  || ' | Rs. ' || TO_CHAR(r.total_amount, 'FM99,99,990.00'));
 22  v_count := v_count + 1;
 23  v_sum   := v_sum + r.total_amount;
 24  END LOOP;
 25
 26  DBMS_OUTPUT.PUT_LINE(RPAD('-', 62, '-'));
 27  DBMS_OUTPUT.PUT_LINE(v_count || ' bill(s) pending, total Rs. '
 28  || TO_CHAR(v_sum, 'FM99,99,990.00'));
 29  END sp_pending_bills;
 30  /

Procedure created.
```

Figure 62: *Procedure sp_pending_bills – parameterised cursor FOR loop with running totals*

```
SQL> EXEC sp_emergency_queue;
EMERGENCY WAITING LIST (highest priority first)
------------------------------------------------------------------------
1. [T1] Critical   | Visit 418 | Divya Krishnan | arrived 20:50
Complaint: Severe allergic reaction | Dr. Arvind Menon
2. [T2] Very urgent| Visit 416 | Pooja Menon | arrived 20:25
Complaint: High fever and vomiting | Dr. Arvind Menon
3. [T4] Minor      | Visit 417 | Rahul Verma | arrived 20:40
Complaint: Minor ankle sprain | Dr. Vikram Singh

PL/SQL procedure successfully completed.
```

Figure 63: *Output of EXEC sp_emergency_queue – patients ordered by triage priority*

5.2 Triggers

Trigger	Behaviour
trg_reduce_stock	BEFORE INSERT/DELETE on PRESCRIPTION: reduces medicine stock for a new prescription (error ORA-20060 if stock is insufficient) and restores it when a prescription is deleted.
trg_audit_visit	AFTER INSERT/UPDATE/DELETE on VISIT: logs new visits, status changes and deletions.
trg_audit_bill	AFTER INSERT/UPDATE/DELETE on BILL: logs bill generation and payment-status changes.
trg_audit_patient	AFTER INSERT/UPDATE/DELETE on PATIENT: logs registration, updates and deletion.

Trigger	Behaviour
trg_doctor_status	Marks a doctor Busy when a visit becomes Under Treatment and Available again when that treatment ends.
trg_protect_audit	BEFORE UPDATE/DELETE on AUDIT_LOG: blocks any modification, making the log tamper-proof (ORA-20050).

```
SQL> CREATE OR REPLACE PROCEDURE sp_write_audit (
  2  p_table   IN VARCHAR2,
  3  p_action  IN VARCHAR2,
  4  p_record  IN NUMBER,
  5  p_desc    IN VARCHAR2
  6  )
  7  IS
  8  BEGIN
  9  INSERT INTO audit_log (log_id, table_name, action_type, record_id, description)
 10  VALUES (seq_audit.NEXTVAL, p_table, p_action, p_record, SUBSTR(p_desc, 1, 200));
 11  END sp_write_audit;
 12  /

Procedure created.
```

Figure 64: *Helper procedure sp_write_audit used by the audit triggers*

```
SQL> CREATE OR REPLACE TRIGGER trg_reduce_stock
  2  BEFORE INSERT OR DELETE ON prescription
  3  FOR EACH ROW
  4  DECLARE
  5  v_stock  medicine.stock_quantity%TYPE;
  6  v_name   medicine.medicine_name%TYPE;
  7  BEGIN
  8  IF INSERTING THEN
  9  SELECT stock_quantity, medicine_name
 10  INTO v_stock, v_name
 11  FROM medicine
 12  WHERE medicine_id = :NEW.medicine_id
 13  FOR UPDATE;
 14
 15  IF v_stock < :NEW.quantity THEN
 16  RAISE_APPLICATION_ERROR(-20060, 'Insufficient stock for ' || v_name
 17  || ': available ' || v_stock || ', requested ' || :NEW.quantity);
 18  END IF;
 19
 20  UPDATE medicine
 21  SET stock_quantity = stock_quantity - :NEW.quantity
 22  WHERE medicine_id = :NEW.medicine_id;
 23  ELSE                                          -- DELETING
 24  UPDATE medicine
 25  SET stock_quantity = stock_quantity + :OLD.quantity
 26  WHERE medicine_id = :OLD.medicine_id;
 27  END IF;
 28  END trg_reduce_stock;
 29  /

Trigger created.
```

Figure 65: *Trigger trg_reduce_stock – stock control on prescriptions*

```
SQL> CREATE OR REPLACE TRIGGER trg_audit_visit
  2  AFTER INSERT OR UPDATE OR DELETE ON visit
  3  FOR EACH ROW
  4  BEGIN
  5  IF INSERTING THEN
  6  sp_write_audit('VISIT', 'INSERT', :NEW.visit_id,
  7  'New ' || :NEW.visit_type || ' visit for patient ' || :NEW.patient_id);
  8  ELSIF UPDATING THEN
  9  sp_write_audit('VISIT', 'UPDATE', :NEW.visit_id,
 10  'Status ' || :OLD.visit_status || ' -> ' || :NEW.visit_status);
 11  ELSE
 12  sp_write_audit('VISIT', 'DELETE', :OLD.visit_id,
 13  'Visit deleted (patient ' || :OLD.patient_id || ')');
 14  END IF;
 15  END trg_audit_visit;
 16  /

Trigger created.
```

Figure 66: *Trigger trg_audit_visit*

```
SQL> CREATE OR REPLACE TRIGGER trg_audit_bill
  2  AFTER INSERT OR UPDATE OR DELETE ON bill
  3  FOR EACH ROW
  4  BEGIN
  5  IF INSERTING THEN
  6  sp_write_audit('BILL', 'INSERT', :NEW.bill_id,
  7  'Bill generated: total ' || :NEW.total_amount
  8  || ' (' || :NEW.payment_status || ')');
  9  ELSIF UPDATING THEN
 10  sp_write_audit('BILL', 'UPDATE', :NEW.bill_id,
 11  'Payment status ' || :OLD.payment_status || ' -> ' || :NEW.payment_status);
 12  ELSE
 13  sp_write_audit('BILL', 'DELETE', :OLD.bill_id, 'Bill deleted');
 14  END IF;
 15  END trg_audit_bill;
 16  /

Trigger created.
```

Figure 67: *Trigger trg_audit_bill*

```
SQL> CREATE OR REPLACE TRIGGER trg_audit_patient
  2  AFTER INSERT OR UPDATE OR DELETE ON patient
  3  FOR EACH ROW
  4  BEGIN
  5  IF INSERTING THEN
  6  sp_write_audit('PATIENT', 'INSERT', :NEW.patient_id, 'New patient registered');
  7  ELSIF UPDATING THEN
  8  sp_write_audit('PATIENT', 'UPDATE', :NEW.patient_id, 'Patient details updated');
  9  ELSE
 10  sp_write_audit('PATIENT', 'DELETE', :OLD.patient_id, 'Patient record deleted');
 11  END IF;
 12  END trg_audit_patient;
 13  /

Trigger created.
```

Figure 68: *Trigger trg_audit_patient*

```
SQL> CREATE OR REPLACE TRIGGER trg_doctor_status
  2  AFTER INSERT OR UPDATE OF visit_status ON visit
  3  FOR EACH ROW
  4  BEGIN
  5  IF :NEW.visit_status = 'Under Treatment' THEN
  6  UPDATE doctor SET availability_status = 'Busy'
  7  WHERE doctor_id = :NEW.doctor_id;
  8  ELSIF UPDATING AND :OLD.visit_status = 'Under Treatment' THEN
  9  UPDATE doctor SET availability_status = 'Available'
 10  WHERE doctor_id = :NEW.doctor_id;
 11  END IF;
 12  END trg_doctor_status;
 13  /

Trigger created.
```

Figure 69: *Trigger trg_doctor_status – automatic doctor availability*

```
SQL> CREATE OR REPLACE TRIGGER trg_protect_audit
  2  BEFORE UPDATE OR DELETE ON audit_log
  3  BEGIN
  4  RAISE_APPLICATION_ERROR(-20050, 'AUDIT_LOG entries cannot be modified or deleted');
  5  END trg_protect_audit;
  6  /

Trigger created.
```

Figure 70: *Trigger trg_protect_audit – audit entries cannot be changed or deleted*

```
SQL> SELECT log_id, table_name, action_type, record_id, description
  2  FROM audit_log
  3  WHERE log_id > 12
  4  ORDER BY log_id;

no rows selected
```

Figure 71: *Audit log check for entries newer than log_id 12 (none recorded yet at this point in the session)*